متن ارائه‌ی آزمایش ۵ — ضرب‌کننده‌ی بوث

حدود ۵ دقیقه | پوریا عرفانزاده، فاطیما تیمارچی، آیه صابری

0:00 • شروع

سلام. من پوریا عرفانزاده هستم، با فاطیما تیمارچی و آیه صابری. آزمایش پنجم ما طراحی یه ضرب‌کننده‌ی بوث بود.

0:20 • صورت مسئله

باید دو تا عدد علامت‌دار رو با الگوریتم بوث ضرب کنیم. دستور کار دو تا قید گذاشته بود: اوّل اینکه مسیر داده و واحد کنترل باید جدا طراحی بشن و بعد به هم وصل بشن، دوم اینکه شیفت‌دهنده باید بتونه تو هر پالس کلاک بیشتر از یه بیت جابه‌جا کنه تا نسبت به ضرب معمولیِ شیفت و جمع تسریع داشته باشیم. چیزی که ساختیم یه ضرب‌کننده‌ی چهاربیتی علامت‌داره تو متمّم دو، با حاصل‌ضرب هشت‌بیتی.

0:50 • خود الگوریتم بوث

بوث سه تا رجیستر داره: انباره‌ی A که اوّلش صفره، Q که مضروب‌فیه رو نگه می‌داره، و M که مضروب رو. یه تک‌بیت کمکی هم سمت راست Q می‌ذاریم.

تو هر تکرار به زوجِ آخرین بیت Q و همون بیت کمکی نگاه می‌کنیم: اگه صفر و یک بود، M رو جمع می‌کنیم؛ اگه یک و صفر بود، کم می‌کنیم؛ و اگه هر دو یکسان بودن، هیچ کاری نمی‌کنیم. بعدش کلّ مجموعه رو یه بیت شیفت حسابی به راست می‌دیم — حسابی یعنی بیت علامت تکرار می‌شه.

چرا این کار می‌کنه؟ چون یه دنباله‌ی پیوسته از یک‌ها تو مضروب‌فیه رو می‌شه به‌جای چند تا جمع، فقط با یه تفریق در اوّل دنباله و یه جمع در آخرش حساب کرد. زوج «یک و

صفر» دقیقاً آغاز چنین دنباله‌ای رو علامت می‌زنه و زوج «صفر و یک» پایانش رو. مزیّت دیگه‌ش هم اینه که اصلاً لازم نیست علامت عملوندها رو جداگانه مدیریت کنیم.

1:45 • شیفت چندبیتی

حالا قسمت تسریع. تو نسخه‌ی پایه، حتّی وقتی زوج بیت‌ها یکسانه و هیچ عمل حسابی لازم نیست، بازم یه پالس کلاک صرف یه شیفت یک‌بیتی می‌شه. ایده اینه که چند تا بیت همسانِ کنار هم رو یه‌جا رد کنیم.

برای اینکه بفهمیم چند تا بیت می‌شه رد کرد، Q رو با نسخه‌ی یه‌بیت شیفت‌خورده‌ی خودش XOR می‌کنیم. نتیجه یه نقشه‌ست که هر بیت روشنش یعنی «اینجا بیت‌های مجاور فرق دارن». جایگاه اوّلین بیت روشن از سمت راست، دقیقاً طول دنباله‌ی همسانِ انتهای Qه.

دو تا نکته‌ی پیاده‌سازی داشت. بیت پرارزش این نقشه رو به‌اجبار یک می‌کنیم، وگرنه برای Q برابر با «همه صفر» یا «همه یک» اصلاً بیت روشنی نداشتیم. و اندازه‌ی شیفت رو با تعداد بیت‌های باقی‌مونده مینیمم می‌گیریم، وگرنه شیفت آخر می‌تونست از مرز چهار بیت رد بشه.

نتیجه اینه که تعداد تکرارها به‌جای چهارِ ثابت، بین یک تا چهار متغیّره. برای این کار به یه barrel shifter نیاز داریم که اندازه‌ی شیفتش در زمان اجرا تعیین می‌شه.

2:45 • بیت نگهبان

یه نکته‌ی ظریف که به نظرم مهم‌ترین جزئیّات طراحیه. تو گام تفریق باید A منهای M رو حساب کنیم. حالا اگه M برابر منفی ۸ باشه، یعنی کوچک‌ترین عدد چهاربیتی علامت‌دار، این تفریق می‌شه A به‌علاوه‌ی ۸ —— و مثبت ۸ اصلاً تو چهار بیت علامت‌دار جا نمی‌شه.

برای همین انباره و رجیستر مضروب رو پنج‌بیتی گرفتیم و مضروب موقع لود شدن با بسط

علامت به پنج بیت میاد. این بیت اضافی همون بیت نگهبانه و فقط تو واسط نهایی کنار گذاشته می‌شه.

3:20 • معماری و تست

طرح سه تا ماژول داره: مسیر داده، واحد کنترل، و یه ماژول بالا که فقط این دو تا رو به هم می‌دوزه. واحد کنترل یه FSM چهار حالته‌ست: IDLE، ARITH، SHIFT و DONE.

یه نکته‌ی مهم تو اتّصالشون هست: مقدار جاری Q از مسیر داده برمی‌گرده به واحد کنترل. چرا؟ چون تو هر گام باید طول دنباله‌ی همسان رو روی مقدار شیفت‌خورده‌ی *فعلی* حساب کنیم، نه روی ورودی اوّلیّه. این بازخورد هسته‌ی کلّ سازوکار شیفت متغیّره.

تست‌بنچ فراگیره — هر ۱۶ مقدار ممکن مضروب رو در هر ۱۶ مقدار مضروب‌فیه ضرب می‌کنه، یعنی هر ۲۵۶ حالت، و نتیجه رو با عملگر ضرب خود Verilog مقایسه می‌کنه. با این پوشش کامل، حالت‌های مرزی مثل صفر و منفی یک و مخصوصاً منفی ۸ خودبه‌خود پوشش داده می‌شن. هر ۲۵۶ تا هم پاس شدن.

تو شکل موج‌ها هر دو حدّ زمانی رو دنبال کردیم. بهترین حالت وقتیه که مضروب‌فیه ۱۱۱۱ باشه: هر چهار بیت تو یه پالس مصرف می‌شن و کلّ ضرب با یه تکرار تموم می‌شه، یعنی تسریع چهاربرابر. بدترین حالت وقتیه که ۰۱۰۱ باشه؛ اونجا بیت‌ها یکی در میون عوض می‌شن، اندازه‌ی شیفت همیشه یک می‌مونه و مدار دقیقاً مثل الگوریتم پایه چهار تکرار می‌کنه — یعنی درست کار می‌کنه ولی مزیّت زمانیش رو از دست می‌ده.

4:25 • سنتز و جمع‌بندی

سنتز: ۸۸ عنصر منطقی، ۲۰ رجیستر و ۱۹ پین، همه زیر یک درصد ظرفیّت قطعه. دو تا نکته تو گزارش بود. اوّل اینکه تعداد ضرب‌کننده‌های توکار مصرف‌شده صفره، یعنی سنتزکننده هیچ بلاک ضرب آماده‌ی تراشه رو برنداشته و کلّ ضرب واقعاً با همین مسیر

داده‌ی خودمون انجام می‌شه. دوم اینکه ثبات حالت رو سه‌بیتی تعریف کرده بودیم، ولی چون فقط چهار مقدار می‌گیره ابزار بیت پرارزشش رو حذف کرده.

هر ۱۹ پین رو هم با Pin Planner نسبت دادیم: دو تا عملوند رو هشت کلید کشویی، شروع رو کلید نهم، کلاک رو نوسان‌ساز ۵۰ مگاهرتزی برد، حاصل‌ضرب رو هشت چراغ قرمز و valid رو یه چراغ سبز. Fmax گزارش‌شده ۱۶۳ مگاهرتزه که هم از ۵۰ مگاهرتز برد و هم از ۱۰۰ مگاهرتز شبیه‌سازی حاشیه‌ی کافی داره. چون آزمایشگاه مجازی بود، رو برد پروگرام نکردیم.

جمع‌بندی: جداسازی مسیر داده و کنترل که هم خواسته‌ی دستور کار بود و هم خوندن و آزمودن طرح رو ساده کرد، تعمیم شیفت یک‌بیتی به چندبیتی که تعداد تکرارها رو از چهارِ ثابت به یک تا چهار رسوند، و یه بیت نگهبان که حالت مرزی منفی ۸ رو نجات داد. ممنون، اگه سؤالی هست در خدمتم.